

## 第六章 模型预训练

在前述章节中已经详细介绍了预训练的数据准备（第 4 章）与模型架构（第 5 章）。本章将主要讨论如何进行大语言模型的预训练（第 6.1 节）。首先，将介绍文本建模的预训练任务（第 6.1 节），然后针对大模型的场景介绍训练优化设置（第 6.2 节）和高效可扩展的训练技术（第 6.3 节），最后给出效率分析（第 6.4 节）和相应的代码实践（第 6.5 节）。

### 6.1 预训练任务

在进行模型的大规模预训练时，往往需要设计合适的自监督预训练任务，使得模型能够从海量无标注数据中学习到广泛的语义知识与世界知识。目前，常用的预训练任务主要分为三类，包括语言建模（Language Modeling, LM）、去噪自编码（Denoising Autoencoding, DAE）以及混合去噪器（Mixture-of-Denoisers, MoD）。图 6.1 展示了这三种任务各自的输入与输出示例。

#### 6.1.1 语言建模

语言建模任务是当前绝大部分大语言模型广泛采用的预训练任务。该任务的核心在于“预测下一个词元”，并且经常被应用于训练基于解码器的大语言模型，例如 GPT-3 [23] 和 PaLM [33] 等。形式化来说，给定一个词元序列  $\mathbf{u} = \{u_1, \dots, u_T\}$ ，语言建模任务的目标定义为词元的预测任务：基于序列中当前位置之前的词元序列  $\mathbf{u}_{<t}$ ，采用自回归的方式对于目标词元  $u_t$  进行预测。在训练过程中，模型通常根据以下的似然函数进行优化：

$$\mathcal{L}_{\text{LM}}(\mathbf{u}) = \sum_{t=1}^T \log P(u_t | \mathbf{u}_{<t}). \quad (6.1)$$

可以发现，语言建模任务与人类生成语言数据（如口语表达、书面写作等）的方式存在相似之处，都是基于前序内容生成（或预测）后续的内容。尽管这种对下一个词元的预测看似简单，但当预训练数据足够丰富时，大语言模型便能够学习到自然语言的生成规律与表达模式。正如 Ilya Sutskever 在接受黄仁勋采访的时候

给出的解释<sup>1</sup>，通过对词元更精准的预测，模型就可以更好地理解文本、建模世界语义知识。该访谈实录原文摘录如例 6.1 所示<sup>2</sup>。

Say you read a detective novel. It's like complicated plot, a storyline, different characters, lots of events, mysteries like clues, it's unclear. Then, let's say that at the last page of the book, the detective has gathered all the clues, gathered all the people and saying, "okay, I'm going to reveal the identity of whoever committed the crime and that person's name is". Predict that word. ...

Now, there are many different words. But predicting those words better and better, the understanding of the text keeps on increasing. GPT-4 predicts the next word better.

### 例 6.1 Ilya Sutskever 对于预测下一个词元任务有效性的解释

此外，从本质上看，基于语言建模的预训练还可以看作是一种多任务学习过程。例如，在预测句子前缀“这部电影剧情饱满，演员表演得也很棒，非常好看”中的“好看”时，模型实际上在进行情感分析任务的语义学习；而在预测句子前缀“小明有三块糖，给了小红两块糖，还剩下一块糖”中的“一块糖”时，则是在进行数学算术任务的语义学习。可以列举出来更多类似的例子，覆盖更广的任务范围。因此，基于大规模文本语料的预训练任务能够潜在地学习到解决众多任务的相关知识能力。

语言建模的一个重要变种是前缀语言建模（Prefix Language Modeling）任务，这种任务专门为采用前缀解码器架构（详细介绍见第 5.3.3 节）的模型而设计。在训练阶段，每个文本序列  $\mathbf{u}$  会根据随机选择的位置  $k(1 \leq k \leq T)$  切分为前缀  $\mathbf{u}_{\text{prefix}} = \{u_1, \dots, u_k\}$  和后缀  $\mathbf{u}_{\text{suffix}} = \{u_{k+1}, \dots, u_T\}$  两个部分。与标准语言建模任务不同，在前缀解码器中，仅后缀中的词元损失会被计入总损失。该任务的训练目标函数可以形式化地表示为：

$$\mathcal{L}_{\text{Prefix}}(\mathbf{u}) = \log P(\mathbf{u}_{\text{suffix}} | \mathbf{u}_{\text{prefix}}) = \sum_{t=k+1}^T \log P(u_t | \mathbf{u}_{<t}). \quad (6.2)$$

可以看到，前缀语言建模任务本质上是基于前缀信息来预测后缀的词元，这与自然语言处理任务中常见的基于输入来预测输出的模式十分相似。然而，在模型预训练阶段的损失函数中，由于并未将所有词元的损失都纳入计算，当使用相同规

<sup>1</sup><https://www.nvidia.com/en-us/on-demand/session/gtc-spring23-S52092/>

<sup>2</sup><https://lifearchitect.ai/ilya/>

模的数据集进行训练时，采用前缀语言建模训练的模型在性能上通常会稍逊于使用标准语言建模任务训练的模型 [191]。

语言建模的另一个重要变种是中间填充任务 [192]。此任务通过重新调整输入序列的顺序，旨在训练模型对于中间缺失信息的填充能力。具体来说，一个输入序列  $\mathbf{u}$  被划分为三个部分：前缀  $\mathbf{u}_{\text{prefix}}$ 、中间部分  $\mathbf{u}_{\text{middle}}$  和后缀  $\mathbf{u}_{\text{suffix}}$ 。随后，中间部分被移至序列末尾。因此，模型需要自回归地对新序列  $\mathbf{u}_{\text{prefix}} \oplus \mathbf{u}_{\text{suffix}} \oplus \mathbf{u}_{\text{middle}}$  进行预测。通过这种方式，模型能够学习填充中间缺失信息的能力。这种任务的训练函数可表示如下：

$$\mathcal{L}_{\text{FIM}}(\mathbf{u}) = \log P(\mathbf{u}_{\text{prefix}}) + \log P(\mathbf{u}_{\text{suffix}}|\mathbf{u}_{\text{prefix}}) + \log P(\mathbf{u}_{\text{middle}}|\mathbf{u}_{\text{prefix}}, \mathbf{u}_{\text{suffix}}). \quad (6.3)$$

通常来说，中间填充任务被用作标准语言建模方法的辅助任务。在保留预测下一个词能力的同时，这种方法使得模型具备对于文本中间部分内容的恢复能力。这种预训练任务经常被用于训练代码预训练模型，从而提升模型在代码补全等实际应用场景中的表现。

对于语言建模任务，这里以 Transformers 库中 LLaMA 模型的前向计算的代码为例，说明损失函数的计算过程。前文已经展示了 LLaMA 模型的详细配置 LlamaModel，Transformers 库中往往将模型的预测头单独封装，例如语言建模任务使用的 LlamaForCausalLM 就是封装了映射到词表的参数矩阵 `self.lm_head`。

```

1 class LlamaForCausalLM(LlamaPreTrainedModel):
2     def __init__(self, config):
3         super().__init__(config)
4         self.model = LlamaModel(config)
5         self.vocab_size = config.vocab_size
6         self.lm_head = nn.Linear(config.hidden_size, config.vocab_size,
7                                   ↪ bias=False) # 将最后一层输出映射为词汇表中每个词元的概率
8
9     def forward(
10         self,
11         input_ids: torch.LongTensor = None,
12         attention_mask: Optional[torch.Tensor] = None,
13         position_ids: Optional[torch.LongTensor] = None,
14         labels: Optional[torch.LongTensor] = None,
15         **kwargs,
16     ) -> Union[Tuple, CausalLMOutputWithPast]:
17         outputs = self.model(
18             input_ids=input_ids,
19             attention_mask=attention_mask,
20             position_ids=position_ids,
21         )
22         # 首先，将输入送入 LlamaModel 中获得最后一层的隐状态
23         hidden_states = outputs[0]
24         logits = self.lm_head(hidden_states).float()
25         # 之后，将隐状态送入映射头中转化为词汇表中每个词元的概率

```

```

25
26     loss = None
27     if labels is not None:
28         # 选择出了最后一个词元之外所有预测的概率，并选择出给定的标签中出了第
           ↳ 一个之外的所有标签，两者一一对应
29         shift_logits = logits[..., :-1, :].contiguous()
30         shift_labels = labels[..., 1:].contiguous()
31         # Flatten the tokens
32         loss_fct = CrossEntropyLoss()
33         # 将同批次中不同序列的词元铺平来方便计算
34         shift_logits = shift_logits.view(-1, self.config.vocab_size)
35         shift_labels = shift_labels.view(-1)
36         shift_labels = shift_labels.to(shift_logits.device)
37         # 计算交叉熵损失
38         loss = loss_fct(shift_logits, shift_labels)
39
40     return CausalLMOutputWithPast(
41         loss=loss,
42         logits=logits,
43     )

```

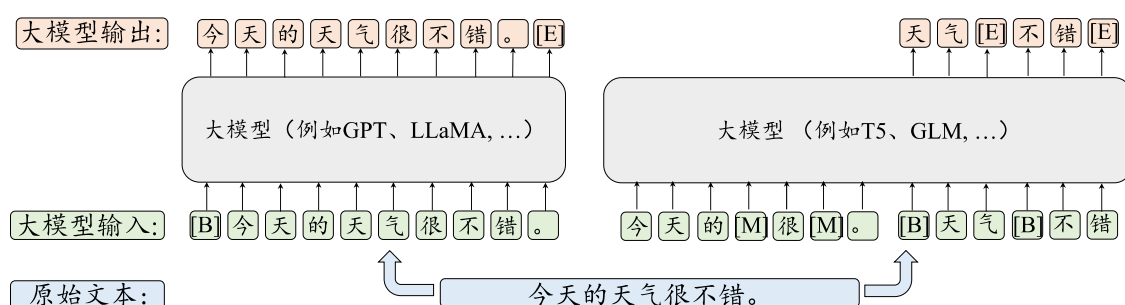


图 6.1 语言建模和去噪自编码的输入输出对比

### 6.1.2 去噪自编码

除了传统的语言建模任务外，去噪自编码任务是另一种常见的语言模型预训练任务，广泛应用于 BERT、T5 等预训练语言模型中 [13, 77]。在去噪自编码任务中，输入文本经过一系列随机替换或删除操作，形成损坏的文本  $u_{\setminus \tilde{u}}$ 。模型的目标是根据这些损坏的文本恢复出被替换或删除的词元片段  $\tilde{u}$ 。去噪自编码器的训练目标可以用以下数学公式表示：

$$\mathcal{L}_{\text{DAE}}(\mathbf{u}) = \log P(\tilde{\mathbf{u}} | \mathbf{u}_{\setminus \tilde{\mathbf{u}}}). \quad (6.4)$$

与语言建模相比，去噪自编码任务的实现更为复杂，需要设定额外的优化策略，如词元替换策略、替换片段长度、替换词元比例等。这些策略的选择会直接



影响模型的训练效果。尽管去噪自编码任务在许多预训练语言模型中得到了广泛应用。然而，相比于语言建模任务，目前完全使用去噪自编码进行预训练的大语言模型还较为有限。代表性的模型包括 FLAN-T5。

### 6.1.3 混合去噪器

混合去噪器，又称 UL2 损失 [193]，通过将语言建模和去噪自编码的目标均视为不同类型的去噪任务，对于预训练任务进行了统一建模。具体来说，混合去噪器定义了三种去噪器：S-去噪器、R-去噪器和 X-去噪器。

S-去噪器与前缀语言建模的目标相同（如公式 6.2 所示），旨在训练模型学习基于给定前缀信息生成合理的后缀文本的能力。相比之下，R-去噪器和 X-去噪器与去噪自编码任务的优化目标更为相似（如公式 6.4 所示）。二者仅仅在被掩盖片段的跨度和损坏比例上有所区别。R-去噪器屏蔽序列中约 15% 的词元，且每个被屏蔽的片段仅包含 3 到 5 个词元。而 X-去噪器则采用更长的片段（12 个词元以上）或更高的损坏比例（约 50%），进而要求模型能够精准还原原始信息。这种设置增加了任务难度，迫使模型学习到更全面的文本表示。

为了引导模型针对不同类型的输入选择相应的去噪器，输入句子会以特殊词元（如 [R], [S], [X]）作为开头。这种标记方式引导模型识别输入中使用的去噪器，并对该去噪器损坏的词元进行还原。

混合去噪器被应用于训练 UL2 [193] 和 PaLM 2 [194] 等大语言模型。

## 6.2 优化参数设置

与传统神经网络的优化类似，通常使用**批次梯度下降算法**来进行模型参数的调优。同时，通过调整**学习率**以及优化器中的**梯度修正策略**，可以进一步提升训练的稳定性。为了防止模型对数据产生过度拟合，训练中还需要引入一系列**正则化方法**。下面将详细介绍适用于大模型场景的训练优化设置。为了方便读者查阅，表 6.1 中汇总了一些常见模型的训练优化设置。

### 6.2.1 基于批数据的训练

在大模型预训练中，通常将**批次大小 (Batch Size)**设置为较大的数值，例如 1M 到 4M 个词元，从而提高训练的稳定性和吞吐量。为了更好地训练大语言模

表 6.1 现有大语言模型的详细优化设置（表格来源：[10]）

| 模型              | 批次大小         | 学习率<br>(预热 → 峰值 → 衰减)           | 优化器       | 精度<br>类型 | 权重<br>衰减 | 梯度<br>裁剪 |
|-----------------|--------------|---------------------------------|-----------|----------|----------|----------|
| GPT-3           | 32K → 3.2M   | 预热 → $6 \times 10^{-5}$ → 余弦    | Adam      | FP16     | 0.1      | 1.0      |
| PanGu- $\alpha$ | -            | $2 \times 10^{-5}$              | Adam      | -        | 0.1      | -        |
| OPT             | 2M           | 预热 → $1.2 \times 10^{-4}$ → 手动  | AdamW     | FP16     | 0.1      | -        |
| PaLM            | 1M → 4M      | $1 \times 10^{-2}$ → 平方根倒数      | Adafactor | BF16     | $lr^2$   | 1.0      |
| BLOOM           | 4M           | 预热 → $6 \times 10^{-5}$ → 余弦    | Adam      | BF16     | 0.1      | 1.0      |
| MT-NLG          | 64K → 3.75M  | 预热 → $5 \times 10^{-5}$ → 余弦    | Adam      | BF16     | 0.1      | 1.0      |
| Gopher          | 3M → 6M      | 预热 → $4 \times 10^{-5}$ → 余弦    | Adam      | BF16     | -        | 1.0      |
| Chinchilla      | 1.5M → 3M    | 预热 → $1 \times 10^{-4}$ → 余弦    | AdamW     | BF16     | -        | -        |
| Galactica       | 2M           | 预热 → $7 \times 10^{-6}$ → 余弦    | AdamW     | -        | 0.1      | 1.0      |
| LaMDA           | 256K         | -                               | -         | BF16     | -        | -        |
| Jurassic-1      | 32k → 3.2M   | 预热 → $6 \times 10^{-5}$         | -         | -        | -        | -        |
| LLaMA-2         | 4M           | 预热 → $1.5 \times 10^{-4}$ → 余弦  | AdamW     | -        | 0.1      | 1.0      |
| Pythia          | 2M           | 预热 → $1.4 \times 10^{-4}$ → 余弦  | Adam      | FP16     | 0.01     | 1.0      |
| Baichuan-2      | -            | 预热 → $1.5 \times 10^{-4}$ → 余弦  | AdamW     | BF16     | 0.1      | 0.5      |
| Qwen-1.5        | 4M           | 预热 → $3 \times 10^{-4}$ → 余弦    | AdamW     | BF16     | 0.1      | 1.0      |
| InternLM-2      | 5M           | 预热 → $3 \times 10^{-4}$ → 余弦    | AdamW     | -        | 0.1      | -        |
| Falcon          | 预热 → 4M      | 预热 → $1.25 \times 10^{-4}$ → 余弦 | AdamW     | BF16     | 0.1      | 0.4      |
| DeepSeek        | 18M          | 预热 → $3.2 \times 10^{-4}$ → 余弦  | AdamW     | BF16     | 0.1      | 1.0      |
| Yi              | 256K         | $1 \times 10^{-5}$              | AdamW     | BF16     | 0.1      | 1.0      |
| YuLan           | 4.5M         | 预热 → $3 \times 10^{-4}$ → 余弦    | Adam      | BF16     | 0.1      | 1.0      |
| GLM-130B        | 0.4M → 8.25M | 预热 → $8 \times 10^{-5}$ → 余弦    | AdamW     | FP16     | 0.1      | 1.0      |
| T5              | 64K          | $1 \times 10^{-2}$ → 平方根倒数      | AdaFactor | -        | -        | -        |

型，现在很多工作都采用了**动态批次调整策略**，即在训练过程中逐渐增加批次大小，最终达到百万级别。例如，GPT-3 的批次大小从 32K 个词元逐渐增加到 3.2M 个词元；PaLM-540B 的批次大小从 1M 个词元逐渐增加到 4M 个词元。相关研究表明，**动态调整批次大小的策略可以有效地稳定大语言模型的训练过程 [33]**。这是因为较小的批次对应反向传播的频率更高，训练早期可以使用少量的数据让模型的损失尽快下降；而较大的批次可以在后期让模型的损失下降地更加稳定，使模型更好地收敛。

### 6.2.2 学习率

现有的大语言模型在预训练阶段通常采用相似的学习率调整策略，包括**预热阶段和衰减阶段**。**预热阶段**一般占整个训练步骤的 0.1% 至 0.5%，然后**学习率**便开始进行衰减。在模型训练的初始阶段，由于参数是随机初始化的，梯度通常也比较大，因此需要使用**较小的学习率使得训练较为稳定**。训练中通常采用线性预

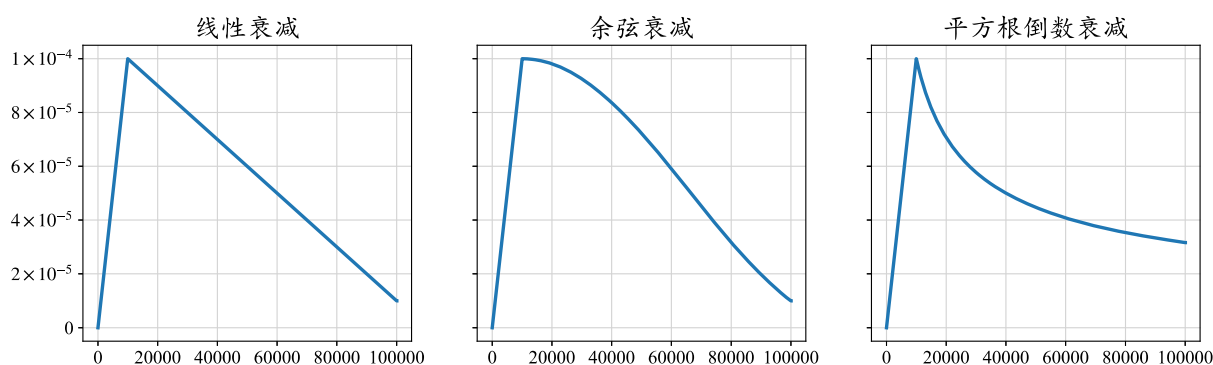


图 6.2 学习率线性衰减、余弦衰减和平方根倒数衰减示意图

热策略来逐步调整学习率。具体来说，学习率将从一个非常小的数值（例如 0 或者  $1 \times 10^{-8}$ ）线性平稳增加，直到达到预设的最大阈值。模型在学习率较大时可以加快收敛速度，这个最大阈值通常设定在  $5 \times 10^{-5}$  到  $1 \times 10^{-4}$  之间。例如，GPT-3 的学习率最大值被设定为  $6 \times 10^{-5}$ ），LLaMA 的学习率最大值被设定为  $1.5 \times 10^{-4}$ 。达到最大阈值之后学习率会开始逐渐衰减，以避免在较优点附近来回震荡。最后，学习率一般会衰减到其最大阈值的 10%。常见的衰减策略有线性衰减，余弦衰减，平方根倒数衰减，它们的学习率变化如图 6.2 所示。

### 6.2.3 优化器

在已有的工作中，大语言模型的训练通常采用 Adam [195] 及其变种 AdamW [196] 作为优化器。Adam 优化器使用梯度的“动量”作为参数的更新方向，它使用历史更新步骤中的梯度加权平均值来代替当前时刻的梯度，从而缓解样本随机性带来的损失震荡。进一步，Adam 使用自适应的学习率方法，通过梯度的加权“二阶矩”对梯度进行修正（可以看做使用“标准差”进行“归一化”），从而防止梯度过小导致模型难以优化。Adam 在优化中引入了三个超参数，在大模型训练中通常采用以下设置： $\beta_1 = 0.9$ ， $\beta_2 = 0.95$  和  $\epsilon = 10^{-8}$ 。此外，谷歌的研究者提出了 Adafactor 优化器 [197]，它是 Adam 优化器的一个变种，通过引入了特殊设计可以在训练过程中节省显存，被用于 PaLM 和 T5 等大语言模型的训练。Adafactor 常见的超参数设置如下： $\beta_1 = 0.9$ ， $\beta_2 = 1.0 - k^{-0.8}$ ，其中  $k$  表示训练步数。

### 6.2.4 稳定优化技术

在大语言模型的训练过程中，经常会遇到训练不稳定的问题。下面介绍几种深度学习中常用的稳定训练技术。

- **梯度裁剪**. 训练中一种常见的现象是损失的突增。为了解决这一问题，可以采取梯度裁剪（Gradient Clipping）的方法，把梯度限制在一个较小的区间内：当梯度的模长超过给定的阈值后，便按照这个阈值进行截断。在大模型训练中，这个阈值通常设置为 1.0。

- **训练恢复**. 为了进一步避免训练过程的异常情况，另一种常用的实践策略是每隔固定的步数设置一些模型存档点。当模型发生了训练异常时（例如损失激增），便可以选择前一个存档点重启训练过程，并跳过可能导致问题的数据。

- **权重衰减**. 在模型的训练过程中也通常会引入正则化技术来稳定训练过程，提高模型的泛化能力。AdamW 中采用了权重衰减（Weight Decay）方法，在每次更新模型参数的时候引入衰减系数，这个系数通常设置为 0.1。

- **Dropout**. 此外，传统深度学习通常采用 Dropout 技术来避免模型过拟合，即在训练中随机将一些神经元的输出值置零来避免过拟合。但是在大规模模型训练中，考虑到大规模的训练数据和模型中存在的归一化结构，已有工作很少使用 Dropout 技术。

## 6.3 可扩展的训练技术

随着模型参数规模与数据规模的不断扩展，如何在有限的计算资源下高效地训练模型已经成为制约大语言模型研发的关键技术挑战。其中，主要面临着两个技术问题：一是如何提高训练效率；二是如何将庞大的模型有效地加载到不同的处理器中。在本节中，我们将介绍几种常见的高效训练技术，包括 3D 并行训练、激活重计算和混合精度训练。

### 6.3.1 3D 并行训练

3D 并行策略实际上是三种常用的并行训练技术的组合，即**数据并行（Data Parallelism）**、**流水线并行（Pipeline Parallelism）**和**张量并行（Tensor Parallelism）**。有的工作也会使用模型并行一词，它同时包括了张量并行和流水线并行。

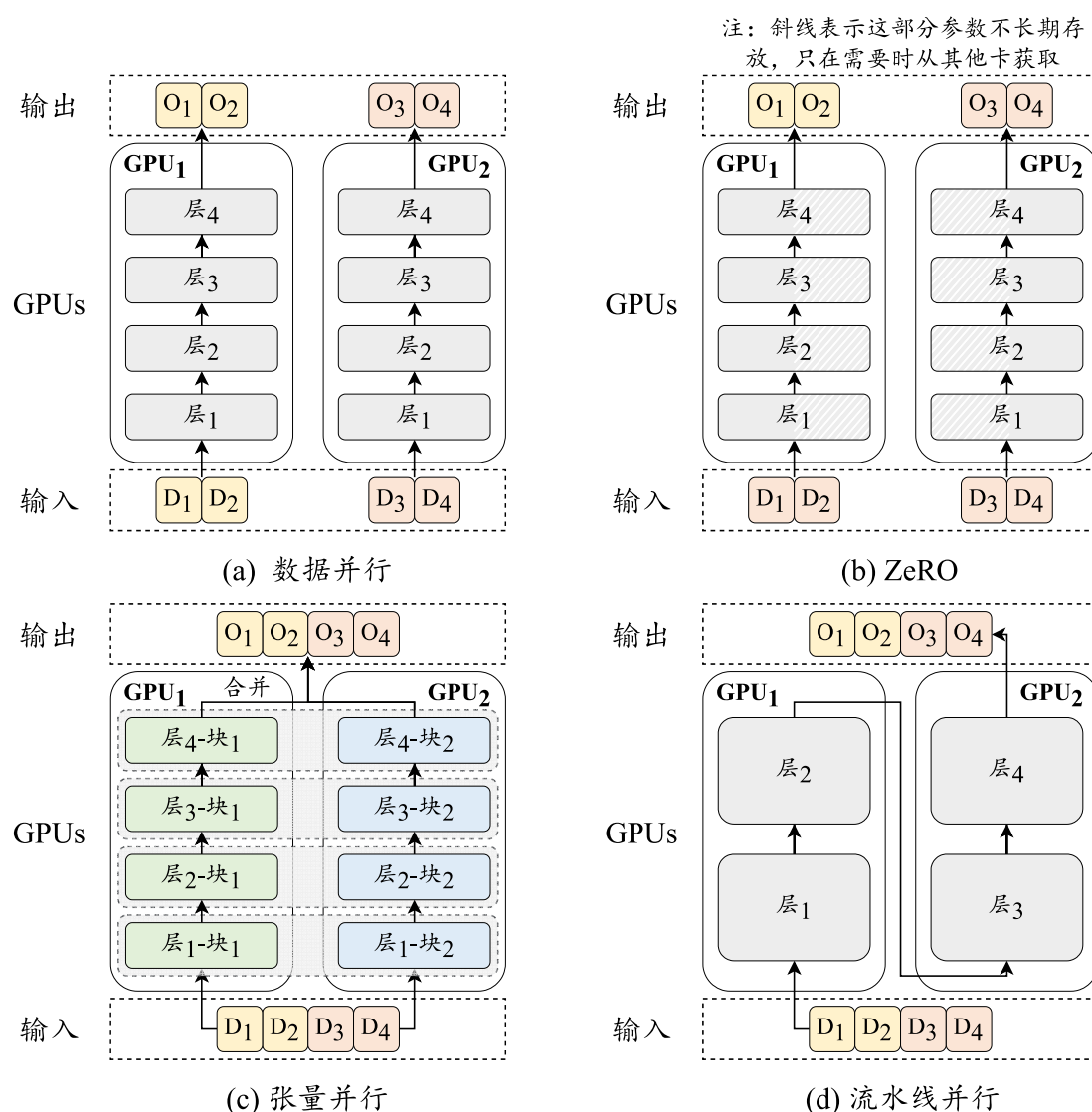


图 6.3 数据并行、ZeRO、张量并行和流水线并行的模型分布情况示意图

- **数据并行.** 数据并行是一种提高训练吞吐量的方法，它将模型参数和优化器状态复制到多个 GPU 上，然后将训练数据平均分配到这些 GPU 上。这样，每个 GPU 只需要处理分配给它的数据，然后执行前向传播和反向传播以获取梯度。当所有 GPU 都执行完毕后，该策略会将不同 GPU 的梯度进行平均，以得到整体的梯度来统一更新所有 GPU 上的模型参数。如图 6.3 (a) 所示，四条数据被分成两份，由两张卡进行分别计算，然后我们会将两张卡的梯度进行平均后再更新模型，这样便等效于执行了批次为 4 的梯度更新。鉴于梯度计算在不同 GPU 上的独立性，数据并行机制展现出高度的可扩展性，可以通过增加 GPU 数量来提高训练效率。数据并行技术的实现相对简便，目前多数深度学习库均已内置了对数据并行策略的支持，例如 TensorFlow 和 PyTorch。

- **流水线并行.** 流水线并行旨在将大语言模型不同层的参数分配到不同的



GPU 上。在实践中，可以将 Transformer 连续的层加载到同一 GPU 上，以减少 GPU 之间传输隐藏状态或梯度的成本。例如，在图 6.3 (d) 中，Transformer 的第 1-2 层部署在 1 号 GPU，将 3-4 层部署在 2 号 GPU。然而，朴素的流水线调度并不能达到真正的并行效果。以图 6.3 (d) 为例，1 号 GPU 在前向传播后需要等待 2 号 GPU 反向传播的结果才能进行梯度传播，因此整个流程是“1 号前向-2 号前向-2 号反向-1 号反向”的串行操作，大大降低了 GPU 的利用率。为了解决这一问题，流水线并行通常需要配合梯度累积 (Gradient Accumulation) 技术进行优化。该技术的主要思想是，计算一个批次的梯度后不立刻更新模型参数，而是累积几个批次后再更新，这样便可以在不增加显存消耗的情况下模拟更大的批次。在流水线并行中使用了梯度累积后，1 号卡前向传播完第一个批次后，便可以不用等待，继续传播第二个和后续的批次，从而提高了流水线的效率。

- 张量并行. 张量并行与流水线并行是两种将大模型参数加载到多个 GPU 上的训练技术。流水线并行侧重于将模型的不同层分配到不同的 GPU 上。相较之下，张量并行的分配粒度更细，它进一步分解了模型的参数张量（即参数矩阵），以便更高效地利用多个 GPU 的并行计算能力。具体地，对于大语言模型中的某个矩阵乘法操作  $WH$ ，参数矩阵  $W$  可以按列分成两个子矩阵  $W_1$  和  $W_2$ ，进而原式可以表示为  $[W_1H, W_2H]$ 。然后，可以将参数矩阵  $W_1$  和  $W_2$  放置在两张不同的 GPU 上，然后并行地执行两个矩阵乘法操作，最后通过跨 GPU 通信将两个 GPU 的输出组合成最终结果。常见的张量并行策略是分解模型注意力层的  $W^Q$ ， $W^K$ ， $W^V$ ， $W^O$  矩阵参数（公式 5.2）和前馈网络层的  $W^U$ ， $W^D$  矩阵参数（公式 5.8）。目前，张量并行已经在多个开源库中得到支持，例如 Megatron-LM [27] 支持对参数矩阵按行按列分块进行张量并行。

### 6.3.2 零冗余优化器

零冗余优化器 (Zero Redundancy Optimizer, ZeRO) 技术由 DeepSpeed 代码库提出，主要用于解决数据并行中的模型冗余问题，即每张 GPU 均需要复制一份模型参数。在图 6.3 (a) 中可以看到，数据并行时每个 GPU 都需要存储大语言模型的相同副本，包括模型参数和优化器参数等。对于每个 GPU，在模型传播到某一层时，其他层的模型和优化器参数并不参与计算，这导致了严重的显存冗余现象，同时也限制了每个 GPU 可以支持的前向传播数据量，降低了训练效率。为了解决这个问题，ZeRO 技术仅在每个 GPU 上保留部分模型参数和优化器参数，当需要时



再从其它 GPU 中读取。如图 6.3 (b) 所示，模型被均分在两张 GPU 上，当需要使用第一层计算时，两张卡分别从对方获取相应的模型参数进行计算，使用完之后便可以释放相应显存，从而降低了显存冗余度。ZeRO 有三种划分模型参数和优化器参数的方案，具体介绍详见第 6.4.4 节。PyTorch 中也实现了与 ZeRO 相似的技术，称为完全分片数据并行 (Fully Sharded Data Parallel, FSDP)。

### 6.3.3 激活重计算

激活重计算 (Activation Recomputation)，也称为梯度检查点 (Gradient Checkpointing)，是一种用于优化反向传播时显存占用的技术。具体来说，给定一个待优化函数  $Y = XW$ ，在反向传播时需要  $X$  的值才能计算  $W$  的导数，所以在前向传播时需要保留这些  $X$  (通常被称为激活值)。然而，保存每一层所有的激活值需要占用大量的显存资源 (具体的显存占用见第 6.4.4 节)。因此，激活重计算技术在前向传播期间仅保留部分的激活值，然后在反向传播时重新计算这些激活值，以达到节约显存的目的，但是同时也会引入额外的计算开销。在大语言模型的训练过程中，激活重计算的常见方法是将 Transformer 的每一层的输入保存下来，然后在反向传播时计算对应层内的激活值。

### 6.3.4 混合精度训练

早期的预训练语言模型 (例如 BERT) 主要使用单精度浮点数 (FP32) 表示模型参数并进行优化计算。近年来，为了训练超大规模参数的语言模型，研发人员提出了混合精度训练 (Mixed Precision Training) 技术，通过同时使用半精度浮点数 (2 个字节) 和单精度浮点数 (4 个字节) 进行运算，以实现显存开销减半、训练效率翻倍的效果。具体来说，为了保证表示精度，需要保留原始 32 位模型的参数副本。但在训练过程中，会先将这些 32 位参数转换为 16 位参数，随后以 16 位精度执行前向传播和反向传播等操作，最后在参数更新时再对 32 位模型进行优化。由于在模型训练中前向传播和反向传播占用了绝大部分优化时间，混合精度训练因而能够显著提升模型的训练效率。常见的半精度浮点数表示方式为 FP16，其包含 1 位符号位、5 位指数位和 10 位尾数位，表示范围为  $-65504$  到  $65504$ 。进一步，谷歌的研究人员深度学习场景提出了新的半精度浮点数表示 BF16，其包含 1 位符号位、8 位指数位和 7 位尾数位，表示范围可以达到  $10^{38}$  数量级。相比于 FP16，BF16 有着更大的数值范围，在大模型训练中被广泛使用。值得一提的是，目前较

为主流的 GPU（例如英伟达 A100）都支持 16 位计算单元运算，因此混合精度训练能够被硬件很好地支持。

## 6.4 模型参数量计算与效率分析

在本节中，我们将介绍如何计算基于 Transformer 架构的大语言模型的参数数量，并给出训练模型时所需要的运算量、训练时间和显存开销估计，方便读者可以估算训练所需要的时间、GPU 显存等计算资源开销。

### 6.4.1 参数量计算

由于当前主流的大模型普遍采用因果解码器架构，因此下面以 LLaMA 模型为范例，深入剖析其参数数量计算方式。对于其他模型，其参数量计算算法可参照此方法计算。首先，假设词表大小为  $V$ ，模型包含  $L$  层解码器，中间状态的维度大小为  $H$ ，前馈网络层的中间状态维度大小为  $H'$ 。我们主要关注计算以下几个部分的参数量：

- 输入嵌入层. 首先，输入嵌入层 ( $E \in \mathbb{R}^{V \times H}$ ) 将词表中的每个单词映射到一个  $H$  维的向量，因此输入编码层有  $VH$  个参数。

- 多头注意力层. 传统的注意力机制部分包含查询 ( $W^Q \in \mathbb{R}^{H \times H}$ )、键 ( $W^K \in \mathbb{R}^{H \times H}$ ) 和值 ( $W^V \in \mathbb{R}^{H \times H}$ ) 的线性变换矩阵，每个变换矩阵都包含  $H^2$  个参数，所以这部分需要  $3 \times H^2$  个参数。同时还需要一个额外的线性变换来将多头注意力机制的输出拼接后映射成最终输出 ( $W^O \in \mathbb{R}^{H \times H}$ )，这又需要  $H^2$  个参数。因此，多头注意力层总共需要  $4 \times H^2$  个参数。

- 前馈网络层. LLaMA 的前馈网络层由三个线性变换组成，中间有一个非线性激活函数。前两个线性变换 ( $W^U \in \mathbb{R}^{H \times H'}$  和  $W^G \in \mathbb{R}^{H \times H'}$ ) 将输入从  $H$  维映射到  $H'$  维，需要  $2 \times HH'$  个参数；最后一个线性变换 ( $W^D \in \mathbb{R}^{H' \times H}$ ) 将输出从  $H'$  维映射回  $H$  维，需要  $HH'$  个参数。因此，前馈网络层总共需要  $3 \times HH'$  个参数。

- 归一化层. 每一层解码器还包含两个 RMSNorm 操作，分别用于对多头注意力层和前馈网络层的输入进行归一化处理，共需要  $2 \times H$  个参数。此外，最后一层的输出也需要进行归一化处理，这又需要额外的  $H$  个参数。

- 输出层. 最后，LLaMA 的输出层包含一个线性变换 ( $W^L \in \mathbb{R}^{H \times V}$ )，将解码器的输出映射到词表大小  $V$  的维度上，使用 softmax 归一化后预测下一个单词的

概率分布。这个线性变换需要  $VH$  个参数。

综上所述，累积输入嵌入层、输出层和  $L$  层解码器每层的多头注意力层、前馈网络层和归一化层，LLaMA 模型的参数量计算公式为：

$$\text{参数量} = 2VH + H + L \cdot (4H^2 + 3HH' + 2H). \quad (6.5)$$

以 LLaMA (7B) 为例计算其参数量，给定  $V = 32000, L = 32, H = 4096, H' = 11008$ ，将这些值代入上述公式中：

$$\begin{aligned} \text{参数量} &= 2 \times 32000 \times 4096 + 4096 + 32 \times (4 \times 4096^2 + 3 \times 4096 \times 11008 + 2 \times 4096) \\ &= 6,738,415,616. \end{aligned}$$

计算得到的参数量与 LLaMA (7B) 模型的实际参数量完全一致。

### 6.4.2 训练运算量估计

模型训练运算量指的是模型在训练过程中，需要进行的浮点运算次数 (Floating Point Operations, FLOP)。这里的浮点运算包括浮点数的加减乘除运算，以及浮点数的指数函数，对数函数，三角函数等运算操作。使用 Transformer 架构进行训练的运算量主要集中在多头注意力计算和线性变换计算。相比之下，归一化、输出映射和旋转位置编码计算所需的运算量较少，而输入编码层则无需计算，因此后续的分析中省略了这些部分。在分析多头注意力和线性变换的运算量时，我们进一步设定以下参数：模型总参数量为  $P$ ，批处理大小为  $B$ ，输入序列长度为  $T$ ，因此训练词元总数为  $C = BT$ ；多头注意力机制包含  $N$  个头，每个头的维度为  $D$ ，因此和中间状态维度  $H$  满足关系  $H = ND$ 。其它定义与参数量计算一节 6.4.1 保持一致。

#### 小贴士 (矩阵乘法运算量)

矩阵  $A \in \mathbb{R}^{n \times m}$  和矩阵  $B \in \mathbb{R}^{m \times p}$  相乘所需的运算量为  $2nmp$ 。

- 多头注意力. 首先分析多头注意力机制一次计算的运算量。对于批次化的数据，计算得到相应的查询、键和值张量 (公式 5.2),  $Q, K, V \in \mathbb{R}^{B \times T \times H}$ ，考虑到需要进行多头计算，这些张量需要进行拆分和转置，得到  $Q', K', V' \in \mathbb{R}^{B \times N \times T \times D}$ 。在注意力计算中 (公式 5.7)  $Q'K'^T$  的矩阵乘法需要  $2BT^2ND$  次浮点运算；接着，进行标准化操作 ( $\sqrt{D}$  放缩) 需要  $BT^2N$  次浮点运算，softmax 操作需要进行指数、加和、归一化操作，总计  $3BT^2N$  次浮点运算；最后结果与  $V'$  进行矩阵乘法，再需要  $2BT^2ND$

次浮点运算。因此，一次多头注意力计算总的浮点运算量为  $4BT^2ND + 4BT^2N$ 。考虑到后向传播的运算量大致为前向传播的两倍<sup>3</sup>，整个模型中多头注意力运算量可表达为：

$$\text{运算量} = 12 \cdot (BT^2ND + BT^2N) \cdot L = 12CTL \cdot (H + N). \quad (6.6)$$

• 线性变换. 接下来考察线性变换的训练运算量，其包括注意力层中的四个映射（公式 5.2、5.3、5.4 和 5.6）、前馈网络层的变换（公式 5.8）以及输出层映射（公式 5.11）。以前馈网络层中的上映射操作  $X'W^U$  为例，中间状态  $X' \in \mathbb{R}^{B \times T \times H}$ ，上映射矩阵  $W^U \in \mathbb{R}^{H \times H'}$ ，因此其前向传播需要  $2BTHH'$  次浮点运算，反向传播则需要  $4BTHH'$  次浮点运算，总计需要  $6CHH'$  次浮点运算，其中  $C = BT$  为训练的词元总数。可以看到，线性变换的运算量与矩阵参数量相关，因此 Transformer 中所有线性变换部分的运算量公式可表达为：

$$\text{运算量} = 6C \cdot \text{线性变换的参数量}. \quad (6.7)$$

若训练过程中采用了激活重计算技术（第 6.3.3 节），反向传播时需要额外进行一次前向传播，则总运算量将变为：

$$\text{运算量} = 8C \cdot \text{线性变换的参数量}. \quad (6.8)$$

最后，通过对比公式 6.5、6.6 和 6.7，可以发现多头注意力的运算量约为线性变换运算量的  $\frac{T}{6H}$ ，考虑到大模型训练场景下序列长度  $T$  小于等于中间状态维度  $H$ ，因此多头注意力运算量最多为线性变换运算量的  $\frac{1}{6}$ ，其影响相对较小。根据公式 6.5，线性变换的参数量通常占总参数量的 95% 以上。因此，可以直接用参数量  $P$  替换公式 6.7 中的线性变换的参数量。在这种情况下，参数量为  $P$  的模型在  $C$  个词元上进行预训练的总体运算量可以按照下式进行估计：

$$\text{运算量} \approx 6CP. \quad (6.9)$$

进一步，如果使用了激活重计算技术，则运算总量约为  $8CP$ 。

下面以 LLaMA (7B) 的训练为例介绍运算总量的计算方法。其参数量  $P \approx 6.74 \times 10^9$ 。这里假设训练数据的词元总数均为  $C = 1 \times 10^9$ ，不使用激活重计算技术，那么 LLaMA (7B) 的训练过程中浮点运算总量为  $6 \times 6.74 \times 10^9 \times 10^9 \approx 4.04 \times 10^{19}$ 。对于 BERT Large 而言，它的参数量为 330M，因此训练所需要的浮点运算总量为  $6 \times 3.36 \times 10^8 \times 10^9 \approx 2.02 \times 10^{18}$ 。

<sup>3</sup>考虑到 Transformer 结构中大多数运算为二元运算（如两个矩阵相乘），需要分别计算损失对两个矩阵的梯度，因此需要两倍的运算量。

### 6.4.3 训练时间估计

在训练过程中，训练时间的计算涉及多个部分，主要包括浮点数运算、数据读写以及多进程同步等。其中，浮点数运算的耗时是训练过程中最主要的部分。因此，可以根据训练运算量的估计公式（公式 6.9）以及 GPU 的浮点运算能力来大致估算训练时间。具体的估计公式如下：

$$\text{训练时间} = \frac{\text{运算量}}{\text{GPU 数量} \times \text{GPU 每秒浮点运算数}}. \quad (6.10)$$

在这个公式中，GPU 每秒浮点运算数通常是 GPU 理论浮点运算能力的 30% 到 70%，而这一比例通常会受到多种实际因素的影响。以 LLaMA (65B) 的预训练为例，其参数量  $P = 6.5 \times 10^{10}$ ，词元数  $C = 1.4 \times 10^{12}$ ，由于采用了激活重计算技术，其运算量大致为  $8CP = 7.28 \times 10^{23}$ 。它在预训练过程中使用了 2,048 张 A100 GPU，而每张 A100 GPU 每秒最多能进行  $3.12 \times 10^{14}$  次 BF16 浮点数运算<sup>4</sup>。我们假设在训练过程中，每张 GPU 能达到每秒  $2 \times 10^{14}$  次 BF16 浮点数运算的实际性能。根据上述公式，可以计算出 LLaMA (65B) 使用 2,048 张 A100 GPU 在 1.4T 个词元上的训练时间大致为  $1.78 \times 10^6$  秒，即大约为 20.6 天。这个估算结果与论文中公布的 21 天基本一致。

### 6.4.4 训练显存估计

接下来讨论如何估计模型在训练中需要的显存资源占用，主要可以分为三个部分：模型参数与优化器、训练中需要保存的激活值和其他显存占用。下面将分别进行分析。

#### 模型参数与优化器的显存占用

模型参数与优化器的显存占用主要指训练过程中的模型参数、模型梯度和优化器参数的占用。在现有的大模型训练方案中，通常会采用混合精度训练（详见第 6.3.4 节），模型参数和模型梯度通常以 16 位浮点数存储，而 Adam 或 AdamW 优化器则需要额外存储 32 位浮点数的模型参数、动量参数以及动量二阶矩参数。假设模型的参数量为  $P$ ，训练中配备有  $G$  张 GPU，训练的数据并行数为  $N_D$ ，流水线并行数为  $N_P$ ，张量并行数为  $N_T$ 。基于上述定义，模型参数与优化器的显存占用情况分析如下：

<sup>4</sup><https://www.nvidia.com/en-us/data-center/a100/>



- 不使用 *ZeRO* 优化技术. 在这种情况下, 由于一个 16 位浮点数需要 2 字节, 一个 32 位浮点数需要 4 字节, 因此模型参数和模型梯度各需要  $2P$  字节的显存, Adam 优化器的模型参数、动量参数以及动量二阶矩参数则各需要  $4P$  字节的显存。通过对于这些显存占用进行累和, 每张 GPU 上会需要使用  $(2+2+4+4+4) \cdot P = 16P$  字节的显存用于存储模型参数与优化器。

- 使用 *ZeRO* 的优化器参数分区方案 (*ZeRO-1*) . 在此方案下, 会将优化器的参数进行平摊到每张 GPU 上, 而模型参数和模型梯度需要每张显卡各自保留。在这种情况下, 每张 GPU 上会需要  $(2+2) \cdot P + (4+4+4) \cdot P/N_D = 4P + 12P/N_D$  字节的显存用于存储模型参数与优化器。在 GPU 数量足够多的情况下 (即  $N_D$  足够大), 相比于不使用 *ZeRO* 的方案, 存储模型参数与优化器的显存会减少至原来的  $\frac{1}{4}$ 。

- 使用 *ZeRO* 的梯度分区方案 (*ZeRO-2*) . *ZeRO-2* 方案是在 *ZeRO-1* 方案的基础上, 进一步将模型梯度也平摊到每张 GPU 上。所以, 每张 GPU 上会需要使用  $2P + (2+4+4+4) \cdot P/N_D = 2P + 14P/N_D$  字节的显存用于存储模型参数与优化器。与 *ZeRO-1* 的推导方式类似, 在 GPU 数量足够多的情况下, 用于存储模型参数与优化器的显存会减少至原来的  $\frac{1}{8}$ 。

- 使用 *ZeRO* 的参数分区方案 (*ZeRO-3*) . *ZeRO-3* 方案是在 *ZeRO-2* 方案的基础上, 更进一步地将模型参数也平摊到每张 GPU 上。在这种情况下, 每张 GPU 需要使用  $16P/N_D$  字节的显存用于存储模型参数与优化器。相比于不使用的方案, 用于存储模型参数与优化器的显存会减少至原来的  $\frac{1}{N_D}$ 。

- 使用了张量并行和流水线并行的方案. 张量并行和流水线并行与上述四种方案全部兼容, 在这种情况下存储模型参数与优化器的显存, 只需要在上文对应情况的公式的基础上, 额外除以  $N_P \times N_T$  即可得到单张 GPU 上的显存开销。

### 训练激活值的显存占用

在大模型的训练期间, 前向传播中需要保留每层的激活值 (中间状态), 来用于后续反向传播中计算梯度并更新模型参数。本部分将以 LLaMA 为例, 重点分析激活值的显存占用情况。模型超参数的符号表示与前文保持一致。

首先, 考虑不使用张量并行、流水线并行、激活重计算等优化方法时, 单张 GPU 上激活值的显存占用情况。

- 多头自注意力层. 公式 5.2、5.3、5.4 中查询、键和值的线性变换需要保存其输入  $X$ , 共计占用  $2BTH$  字节。公式 5.5 中多头注意力计算需要保存输入的查

询 ( $Q$ )、键 ( $K$ ) 和值 ( $V$ ), 共计  $6BTH$  字节。公式 5.7 中合并多头结果需要保存其输入  $\text{Concat}(\text{head}_1, \dots, \text{head}_H)$ , 共计  $2BTH$  字节。若未使用 FlashAttention 优化, 公式 5.5 中  $QK^T$  的结果也需要保存, 占用  $2BT^2N$  字节; 若使用了 FlashAttention, 则无需此部分开销。

- 前馈网络层. LLaMA 的前馈网络层使用了 SwiGLU 激活函数 (公式 5.23), 需要保存其输入  $X$ , 共计  $2BTH$  字节; 同时也需要分别保存  $W^G X$  和  $W^U X$  的值, 共计  $4BTH'$  字节; 最后, 需要保留 SwiGLU 的输出值, 作为后续线性变换的输入 (即公式 5.8 中的  $\sigma(XW^U)$ ), 共计  $2BTH'$  字节。因此, 该部分总计  $2BTH + 6BTH'$  字节的开销。

- 归一化层. 每层解码器包含两个归一化层, 每个归一化层需保存其输入 (即公式 5.9 中的  $\text{MHA}(X_{l-1})$  和  $\text{FFN}(X'_l)$ ), 共需  $4BTH$  字节。

- 输出层. 模型在经过  $L$  层解码器层后, 还要经过归一化层处理, 需要保存其输入  $2BTH$  字节。然后进行词表映射 (公式 5.11), 需要保存其输入  $Y_L$ , 总共  $2BTH$  字节。进一步, 在计算 softmax 函数时, 需要保存其输入  $W^L Y_L$  的值, 在实践中为了提高 softmax 的精度, 这里通常会将输入转化为 32 位浮点数来进行后续计算, 因此需要保存  $4BTV$  字节。

因此, 在未使用 FlashAttention 的情况下, 整个模型的总激活值占用公式如下:

$$\text{激活值占用} = (16BTH + 6BTH' + 2BT^2N) \cdot L + 4BTH + 4BTV. \quad (6.11)$$

如果使用了 FlashAttention 技术, 则从上式中去除  $2BT^2N$  项即可。

接下来将分别分析采用了流水线并行、张量并行和激活重计算等优化技术时激活值的显存占用情况。为了简化分析流程, 后续假设不使用 FlashAttention 技术; 要获得使用了该技术的显存占用, 类似地从公式中移除  $2BT^2N$  项即可。

- 流水线并行. 流水线并行将 LLaMA 中的每个 Transformer 层平均分配到不同 GPU 上, 因此每个 GPU 只需要保存相应的激活值即可。假设流水线并行的并行数为  $N_P$ , 则激活值占用公式为:

$$\text{激活值占用} = (16BTH + 6BTH' + 2BT^2N) \cdot \frac{L}{N_P} + 4BTH + 4BTV. \quad (6.12)$$

- 张量并行. 在进行张量并行优化时, 多头注意力层和前馈网络层中的线性变换操作可以通过拆分参数矩阵, 进而分配到不同的 GPU 上来并行计算结果, 因此对应的激活值也可以分配到相应的 GPU 上去, 包括多头注意力计算中的  $Q, K, V$ ,  $\text{Concat}(\text{head}_1, \dots, \text{head}_H)$  和  $QK^T$ , 以及前馈网络层中的  $W^G X, W^U X$  和  $\sigma(XW^U)$ 。

但是多头注意力层、前馈网络层和两个归一化层的输入 ( $\text{MHA}(X_{l-1})$  和  $\text{FFN}(X'_l)$ ) 无法进行拆分, 每张 GPU 都需要进行保存。假设张量并行的并行数为  $N_T$ , 则激活值占用公式为:

$$\text{激活值占用} = ((8 + \frac{8}{N_T})BTH + \frac{6}{N_T}BTH' + \frac{2}{N_T}BT^2N) \times L + 4BTH + 4BTV. \quad (6.13)$$

- 激活重计算. 在 Transformer 大模型的训练中, 激活重计算在前向传播时仅保存 Transformer 每一层的输入和最后层 softmax 函数的输入, 在反向传播时按需重新计算激活值来减少显存使用。因此在这种情况下, 激活值占用简化为:

$$\text{激活值占用} = (4 + 2L)BTH + 4BTV. \quad (6.14)$$

### 其他显存占用

除了上述主要的显存占用因素外, 显存的消耗还主要来自以下几个方面:

- 代码库内核. PyTorch 框架在加载其自身的代码库内核时, 大约会占用 0.8GB 至 1GB 的显存。这是框架运行所必需的基本开销。

- ZeRO 优化技术实现. 当使用 DeepSpeed 库中的 ZeRO 优化技术时, 显存占用会在 1GB 到 4GB 之间浮动。具体占用量取决于采用的 ZeRO 优化方案等级和相关参数的设置。这部分显存主要用于优化训练过程中的显存管理和通信效率。

- 训练过程中的中间结果和显存碎片. 在计算公式 5.11 的 softmax 函数时, Transformers 的具体实现会额外引入输入两倍的显存开销, 因此需要占用  $8BTV$  字节。此外, 在训练过程中, 由于显存分配和释放的不连续性, 会产生一定的显存碎片, 这些因素通常会导致额外占用 0.5GB 到 1GB 的显存。

### 实例: 训练过程中的总显存占用估计

接下来, 我们将综合运用上述公式, 来估计训练中的显存开销。假设训练中使用了数据并行 (数量为  $N_D$ )、FlashAttention、激活重计算和 ZeRO-3 技术进行效率优化, 并采用了 Transformers 代码库所提供的代码实现, 则每张 GPU 的显存开销为:

$$\text{每张 GPU 显存} \approx \frac{16P}{N_D} + (4 + 2L)BTH + 12BTV + 6. \quad (6.15)$$

- LLaMA (7B) 训练的显存占用. 在 LLaMA (7B) 模型中,  $L = 32, H = 4096$ 。进一步, 假设使用了 2 张 A800 (80G), 批次大小  $B = 8$ , 那么每张 GPU 中的模型参数与优化器的显存占用为  $16P/N_D \approx 5.39 \times 10^{10}$  字节, 约为 50.20GB。在 Transformers 库的实现中, 模型训练激活值的显存占用为  $(4 + 2L) \cdot BTH + 4BTV =$

$(4+2\times 32)\times 8\times 2048\times 4096+4\times 8\times 2048\times 32000\approx 6.66\times 10^9$  字节, 约为 6.20GB; 中间结果的显存占用为  $8BTV=8\times 8\times 2048\times 32000\approx 4.19\times 10^9$  字节, 约为 3.91GB; 再加上每张显卡的其他显存占用部分约 6GB, 两张 A800 (80G) 训练 LLaMA (7B) 时每张 GPU 大约会占用  $50.20+6.20+3.91+6\approx 66GB$ 。

- 不同大小模型需要的显存大小估计. 在上述讨论中, 可以发现模型参数与优化器的显存占用是决定所需训练资源数量的关键。因此在实践中, 至少需要有 16 倍参数数量的显存资源, 才能进行全量的参数训练 (如果资源有限, 读者可以阅读第 7.3 节介绍的轻量化训练方法)。例如, 13B 的模型至少需要  $13\times 16=208GB$  的显存, 因此至少需要 3 张 A800 (80G) 的 GPU, 每张 GPU 剩余约 10GB 显存, 代入公式 6.15 可以再确定批次大小  $B$  最大为 2。但是, 实践中批次大小过小会导致模型训练效率较低, 因此 13B 的模型建议至少使用 4 张 GPU, 代入公式后可以将  $B$  设置为 12 来提高训练效率。基于上述分析, 我们也可以类似地得到训练 30B 和 65B 模型, 推荐至少分别使用 8 张和 16 张 80GB 的 GPU。

## 6.5 预训练代码实践

为了帮助读者更好地理解预训练的过程, 本节将详细展示一个 LLMBBox 和 YuLan-Chat 的预训练示例代码。此示例基于 Transformers 和 DeepSpeed 进行训练。在下面的示例代码中, `train()` 函数涵盖了预训练过程中的主要步骤, 包括模型与分词器的初始化、训练数据的准备等; 然后调用 `Trainer` 类来执行模型训练并保存训练状态。

```

1 from dataclasses import dataclass
2 from dataset.pt_dataset import PTDataset
3 from transformers import (
4     AutoModelForCausalLM,
5     AutoTokenizer,
6     HfArgumentParser,
7     TrainingArguments,
8     Trainer,
9 )
10 from transformers.hf_argparser import HfArg
11
12
13 # 用户输入超参数
14 @dataclass
15 class Arguments(TrainingArguments):
16     # 模型结构
17     model_name_or_path: str = HfArg(
18         default=None,
```

```

19     help="The model name or path, e.g., `meta-llama/Llama-2-7b-hf`",
20 )
21 # 训练数据集
22 dataset: str = HfArg(
23     default="",
24     help="Setting the names of data file.",
25 )
26 # 上下文窗口大小
27 model_max_length: int = HfArg(
28     default=2048,
29     help="The maximum sequence length",
30 )
31 # 只保存模型参数 (不保存优化器状态等中间结果)
32 save_only_model: bool = HfArg(
33     default=True,
34     help="When checkpointing, whether to only save the model, or also
35     ↪ the optimizer, scheduler & rng state.",
36 )
37 # 使用 BF16 混合精度训练
38 bf16: bool = HfArg(
39     default=True,
40     help="Whether to use bf16 (mixed) precision instead of 32-bit.",
41 )
42
43 def train():
44     # 解析命令行参数
45     parser = HfArgumentParser(Arguments)
46     args = parser.parse_args_into_dataclasses()[0]
47     # 加载分词器
48     tokenizer = AutoTokenizer.from_pretrained(
49         args.model_name_or_path,
50         model_max_length=args.model_max_length,
51         padding_side="right",
52         add_eos_token=False,
53     )
54     # 加载模型, 并使用 FlashAttention
55     model = AutoModelForCausalLM.from_pretrained(args.model_name_or_path,
56     ↪ attn_implementation="flash_attention_2")
57     # 初始化训练器、准备训练数据并开始训练
58     kwargs = dict(
59         model=model,
60         args=args,
61         tokenizer=tokenizer,
62         train_dataset=PTDataset(args, tokenizer),
63     )
64
65     trainer = Trainer(**kwargs)
66     trainer.train()
67     trainer.save_model(args.output_dir + "/checkpoint-final")
68     trainer.save_state()
69
70 if __name__ == "__main__":
71     train()

```

其中预训练数据集类 `PTDataset` 的定义如下, `process()` 函数涵盖了预训练



数据的主要处理步骤，包括数据读取、分词、批次化等主要操作。

```

1 import torch
2 from datasets import load_dataset
3 from itertools import chain
4
5
6 class PTDataset:
7
8     def __init__(self, args, tokenizer):
9         self.args = args
10        self.block_size = self.args.model_max_length
11        self.tokenizer = tokenizer
12        self.input_ids = self.process()
13        self.input_ids = self.group_texts(self.input_ids)
14        self.labels = self.input_ids.copy()
15
16        # 数据集长度
17        def __len__(self):
18            return len(self.input_ids)
19
20        # 获取第 i 条数据
21        def __getitem__(self, i):
22            return dict(input_ids=self.input_ids[i], labels=self.labels[i])
23
24        # 数据分词
25        def encode(self, examples):
26            output = self.tokenizer(examples["text"], truncation=True)
27            return output
28
29        # 数据批次化处理
30        def group_texts(self, examples):
31            concatenated_examples = list(chain(*examples))
32            total_length = len(concatenated_examples)
33            if total_length >= self.block_size:
34                total_length = (total_length // self.block_size) *
35                    ↪ self.block_size
36            result = [
37                torch.stack(concatenated_examples[i:i + self.block_size]) for i
38                ↪ in range(0, total_length, self.block_size)
39            ]
40            return result
41
42        # 调用数据集加载、分词、批次化
43        def process(self):
44            input_ids = []
45            list_data_dict = load_dataset('text',
46                ↪ data_files=self.args.dataset)['train']
47            tokenized_dataset = list_data_dict.map(
48                self.encode,
49                batched=True,
50                remove_columns='text',
51            )
52            for example in tokenized_dataset:
53                if len(example['input_ids']) > 0:
54                    input_ids.append(torch.tensor(example['input_ids']))
55            return input_ids

```

进一步，为了实现数据并行训练（单机多卡），我们可以使用 `torchrun` 启动上述代码。具体的运行指令如下面的代码所示。其中，第一行的 `nproc_per_node` 参数用于指定训练的 GPU 数量，`master_port` 参数用于指定训练时通信的端口号。随后的参数都是指定训练时的超参数。例如，`bf16` 用于确定是否使用 BF16 进行训练，`num_train_epochs` 用于指定训练轮数，`per_device_train_batch_size` 用于指定单张 GPU 上的批次大小，`gradient_accumulation_steps` 用于指定梯度累计步数，`learning_rate` 用于指定最大学习率，`warmup_ratio` 用于指定学习率预热的数据比例，`lr_scheduler_type` 用于指定学习率衰减策略，`deepspeed` 用于指定使用 DeepSpeed 的参数文件（例如 ZeRO 策略），`gradient_checkpointing` 用于确定是否使用激活重计算技术。

```

1 torchrun --nproc_per_node=2 --master_port=5999 pretrain.py \
2   --model_name_or_path meta-llama/Llama-2-7b-hf/ \
3   --data_path data/pretrain.json \
4   --bf16 True \
5   --output_dir output/llama-7b \
6   --num_train_epochs 3 \
7   --per_device_train_batch_size 8 \
8   --gradient_accumulation_steps 8 \
9   --evaluation_strategy "no" \
10  --save_strategy "steps" \
11  --save_steps 200 \
12  --save_total_limit 200 \
13  --learning_rate 2e-5 \
14  --weight_decay 0. \
15  --warmup_ratio 0.03 \
16  --lr_scheduler_type "cosine" \
17  --logging_steps 1 \
18  --deepspeed ds_z3_bf16.json \
19  --tf32 True \
20  --gradient_checkpointing True

```

下面将介绍 DeepSpeed 的训练文件，其可以设置 ZeRO 的超参数等参数。这是一个 json 文件，其中 `zero_optimization` 包括了 ZeRO 的训练参数，在这个参数字典里，`stage` 可以设置为 1、2、3 中的一个整数值，用来表示 ZeRO 的划分等级；`stage3_max_live_parameters` 和 `stage3_max_reuse_distance` 是 ZeRO-3 阶段控制模型参数缓存量的超参数，其设置的值越小，占用的显存量越小但通信成本会提高；`stage3_gather_16bit_weights_on_model_save` 用于控制 ZeRO-3 时存档点是否被分片，如果设置为 `false` 则保存时会分片，后续使用时需要使用里面脚本进行合并，否则在保存时不会分片，但在训练保存存档点的时间会变长。

```

1 {
2     "bf16": {
3         "enabled": "auto"
4     },
5     "zero_optimization": {
6         "stage": 3,
7         "overlap_comm": true,
8         "contiguous_gradients": true,
9         "sub_group_size": 1e9,
10        "reduce_bucket_size": "auto",
11        "stage3_prefetch_bucket_size": "auto",
12        "stage3_param_persistence_threshold": "auto",
13        "stage3_max_live_parameters": 1e9,
14        "stage3_max_reuse_distance": 1e9,
15        "stage3_gather_16bit_weights_on_model_save": true
16    },
17    "gradient_accumulation_steps": "auto",
18    "gradient_clipping": "auto",
19    "steps_per_print": 2000,
20    "train_batch_size": "auto",
21    "train_micro_batch_size_per_gpu": "auto",
22    "wall_clock_breakdown": false
23 }

```

以上代码是基于 Transformers 和 DeepSpeed 库实现的简易训练代码，仅使用了数据并行和 ZeRO 优化技术，主要适用于模型大小不超过 30B 且 GPU 数量少于 16 张的场景。如果希望在更多并行计算资源上训练参数量更大的模型，例如在 128 张甚至 2,048 张 GPU 上训练 70B 及以上大小的模型，则需要使用完整的 3D 并行技术（即数据并行、流水线并行和张量并行）来进一步提升训练效率。目前，Megatron-LM 提供了较好的 3D 并行训练支持（详细介绍见第 3.4.3）。此外为了确保训练效率，硬件部署也需要相应的适配，需要使用 NVLink 和 NVSwitch 来实现 GPU 之间的高速通信；对于多台机器的训练，则需要配备 InfiniBand 来实现不同机器之间的高性能数据传输。